CONTEXT ENGINEERING · APPENDIX A

Open-source learning resources



Twelve places to practice engineering the systems around the model.

Repository and source review: 2026-10-12. Examples were inspected, not executed.

A working demo is the start of an engineering exercise. Follow a piece of evidence from its source to an answer or action, identify the boundary that could fail, and decide what would expose that failure. These twelve resources give you concrete systems to inspect while reading the book.

The title describes the requested learning collection, not a blanket license claim. It includes permissively licensed projects, Phoenix under ELv2, and public educational material whose reuse license is unconfirmed. Reading a public notebook does not establish permission to redistribute its code or data. The entries link to original material; no tutorial code is reproduced here.

How to Use This Appendix

Tutorials provide exercises. Framework documentation explains supported interfaces. Reference implementations expose design choices. Primary research supports only the claims and conditions actually evaluated in its experiments. None of the tutorial results below establishes architectural superiority.

Choose one relevant lesson and keep a small experiment record: source revision, environment, inputs, expected result, observed result, failure cases, and cost. The reliability takeaways are this book's engineering applications; proposed extensions are not claims that the upstream lesson already tests those cases.

"Local" has three separate meanings here: a local notebook, a local backend, and local model inference. Check all three before assuming an exercise is offline or has no API charges. Setup and compatibility findings below are source inspections, not successful installation or execution reports.

Specific GitHub lesson links are pinned to inspected revisions. The verification notes record update history, licenses, dependency evidence, and review limitations. For later versions, compare the current repository with that snapshot.

Learning Map

Engineering area	Resources	Primary chapter connections
Prompting and Computational Pragmatics	Anthropic; DSPy	4
Retrieval and Context Assembly	Haystack; Microsoft GraphRAG	7, 9
Knowledge Graphs and Semantic Modeling	KG Tutorials; AWS Graph Notebook	8
State, Tool Use, and Orchestration	Hugging Face; LangGraph 101	5, 13
Authorization and Security	OPA; Agentic Security	11
Observability and Evaluation	Phoenix; LLM Observatory	16, 17

Prompting and Computational Pragmatics

Anthropic Prompt Engineering Interactive Tutorial

Source: anthropics/prompt-eng-interactive-tutorial
Type: Educational tutorial.
Maintainer: Anthropic; Amazon contributes the Bedrock variant.

What you will learn: Practice making task instructions, variable input, and example outputs distinguishable. The email exercise makes an output contract concrete; it is an entry point into the book's discussion of pragmatic intent, not a course in computational pragmatics itself.

Book connection: 04.03 — Examples, Instructions, and Structured Outputs.

Recommended lessons:

- Separating Data and Instructions, exercises 4.1–4.3.
- Using Examples, exercise 7.1: Email Formatting via Examples.

Prerequisites: Basic Python, Jupyter/IPython, the `anthropic` SDK, and access to the chosen model. The Bedrock alternative adds AWS credentials, model access, and its own setup.

Local execution and cost: Notebook execution is local; the supplied model calls are remote and require API access with potential usage charges. There is no supplied local inference path.

Reliability engineering takeaway: Change one prompt variable at a time and check held-out inputs for both formatting and meaning. Delimiters help express intent; the host still validates the result.

Limitations: The default-branch lessons date to 2024 and describe Claude 3-era models. Recheck model availability and prefill support. Regex exercise graders are narrow checks. XML tags are not an injection defense, and generated reasoning is not a faithful execution trace. Link to the direct-API lessons rather than copying their code without a confirmed license.

License and update check: No repository-wide license found. `AmazonBedrock/LICENSE` is MIT No Attribution (MIT-0); do not extend that subtree grant to the `Anthropic 1P` notebooks. Latest substantive change identified: 2024-04-06; see the verification record for its scope and later maintenance activity.

DSPy

Source: stanfordnlp/dspy
Type: Framework documentation and implementation, with tutorials; research-associated.
Maintainer: Omar Khattab and DSPy contributors, hosted by stanfordnlp.

What you will learn: Express a language task as signatures and modules, choose a metric, evaluate a retrieval program, and optimize instructions or demonstrations against examples.

Book connection: 04.01 — In-Context Learning.

Recommended lessons:

- RAG tutorial: evaluation, optimization, cost, and saving the program.

Prerequisites: Python and evaluation fundamentals; the inspected package specifies Python >=3.10,<3.16. The RAG lesson also uses datasets, FAISS, and MLflow. Optional dependencies have narrower Python support than the top-level package.

Local execution and cost: The program and MLflow server can run locally. The selected lesson uses OpenAI generation and embeddings, so budget for API calls, including optimization and judging. Local-model providers are an adaptation path, not an offline reproduction verified here.

Reliability engineering takeaway: Keep the optimization set separate from the final evaluation set. Save the program, metric, model, data versions, and total search cost so an improvement can be reproduced.

Limitations: Optimizers optimize the supplied metric and can overfit it. The RAG lesson's model-based SemanticF1 is not an independent truth oracle. Framework documentation demonstrates mechanisms; the DSPy paper and its experiments must be assessed separately. Older "teleprompter" examples may use different APIs.

License and update check: MIT (`LICENSE`). Latest substantive change identified: 2026-10-12; see the verification record for its scope and later maintenance activity.

Retrieval and Context Assembly

Haystack Tutorials

Source: deepset-ai/haystack-tutorials
Type: Educational notebooks for the Haystack framework.
Maintainer: deepset and contributors.

What you will learn: Build an indexing and retrieval pipeline, pass metadata filters into retrieval, and combine lexical and embedding candidates with ranking. Inspect each component's input and output before assembling model context.

Book connection: 07.02 — Context Assembly Pipelines.

Recommended lessons:

- 27: First RAG Pipeline.
- 31: Metadata Filtering.
- 33: Hybrid Retrieval.

Prerequisites: Python, notebooks, and per-notebook Haystack integration packages. Lesson 27 downloads embedding and generation weights; lesson 33 explicitly selects CUDA for its embedders. The root requirements file is publishing tooling, not the complete lesson environment.

Local execution and cost: Lesson 31 uses local in-memory BM25 without paid APIs. Lesson 27 currently offers local Transformers generation plus an optional OpenAI route. Lesson 33 uses local models after downloads; CPU use requires changing its device configuration. Web fallback lesson 36 adds OpenAI and Serper credentials.

Reliability engineering takeaway: Record what retrieval returned and what assembly admitted. As a book extension, remove a required document or vary a filter and record an explicit missing-evidence outcome.

Limitations: In-memory stores do not teach durable indexing or tenant isolation. A user-supplied metadata filter is not authorization. Avoid mixing old Haystack 1.x examples with these component pipelines. The README still points to a removed lesson-28 filename; use verified lesson paths.

License and update check: Apache-2.0 (`LICENSE`). Latest substantive change identified: 2026-09-03; see the verification record for its scope and later maintenance activity.

Microsoft GraphRAG

Source: microsoft/graphrag
Type: Research-associated reference implementation and framework documentation.
Maintainer: Microsoft GraphRAG contributors.

What you will learn: Follow text through extraction, derived graph structure, community summaries, and query-time context. Compare entity-oriented local search with corpus-wide summary retrieval.

Book connection: 09.03 — Graph and Hybrid Retrieval.

Recommended lessons:

- Getting Started: initialize, index, and query a small corpus.
- Local search: entities and supporting text.
- Global search: community reports.

Prerequisites: Python, CLI use, an input corpus, and configured chat and embedding models. The inspected monorepo specifies Python >=3.11,<3.14, while the quickstart still says 3.10–3.12; match the environment to the package release you install.

Local execution and cost: Indexing and queries can run on a local machine, but the documented default uses paid OpenAI or Azure model access. Other model backends require compatible configuration and structured-output support; no all-local reproduction was performed.

Reliability engineering takeaway: Treat extracted entities and summaries as derived evidence. Compare retrieval routes on the same questions and track indexing cost, evidence support, and query cost separately.

Limitations: The README calls this a demonstration, not an officially supported Microsoft offering. Indexing can be expensive, configuration formats change, and extracted graphs can be incomplete or wrong. The built-in basic vector route is not automatically a strong baseline. A tutorial run does not establish graph superiority or enterprise authorization.

License and update check: MIT (`LICENSE`). Latest substantive change identified: 2026-10-08; see the verification record for its scope and later maintenance activity.

Knowledge Graphs and Semantic Modeling

Knowledge Graph Tutorials

Source: kai-vu/KG-tutorials
Type: Educational course materials.
Maintainer: kai-vu, Knowledge Graphs and Semantic Technologies course.

What you will learn: Create identified statements, distinguish asserted triples from entailments, and inspect validation failures. This makes the difference between describing a domain, reasoning over it, and rejecting an invalid record tangible.

Book connection: 08.02 — RDF, OWL, SPARQL, and Shapes.

Recommended lessons:

- RDF: Restaurant Exercise, Part 1.
- OWL: asserted and inferred graphs.
- SHACL: constraint and validation-report exercises.

Prerequisites: Basic Python and graph concepts. The course recommends Python 3.10.13; dependencies include RDFLib, owlrl, owlready2, notebooks, and additional ML packages. The selected SHACL lesson uses GraphDB with SHACL validation enabled.

Local execution and cost: RDF and OWL exercises use local files and libraries without an LLM API. SHACL exercises need a separately installed GraphDB environment or an adapted validator; the notebook suggests browser tools too. External endpoints and dataset downloads require connectivity. GraphDB's distribution terms are separate.

Reliability engineering takeaway: Keep asserted data, inferred data, and validation reports distinct. A graph that passes shape validation can still contain a false or unauthorized assertion.

Limitations: The README's RDFstar link targets a nonexistent file. Package pins are mixed with unpinned dependencies. The SHACL lesson explicitly says its GraphDB setup does not support the advanced SHACL-rules exercise. Public endpoints have changed in the history; local fixtures make the exercise easier to reproduce. Do not assume code or dataset redistribution rights.

License and update check: No repository-wide license found in the tree or README; reuse permission is unconfirmed. Linked datasets have separate provenance and terms. Latest substantive change identified: 2026-03-11; see the verification record for its scope and later maintenance activity.

AWS Graph Notebook

Source: aws/graph-notebook
Type: Notebook integration framework with educational examples.
Maintainer: AWS / Amazon Neptune contributors.

What you will learn: Issue explicit graph queries from a notebook and inspect the paths or bindings returned. Compare a traversal's direction, depth, filters, and duplicate behavior with the question it is meant to answer.

Book connection: 08.03 — Entity Resolution and Relationship Traversal.

Recommended lessons:

- Gremlin Basic Read Queries: friends and friends-of-friends exercises.
- SPARQL Basics: FILTER, OPTIONAL, DISTINCT, and LIMIT.

Prerequisites: Python, Jupyter, `graph-notebook`, and a compatible graph database endpoint. The inspected package specifies Python >=3.9,<3.14 and JupyterLab 4; its README also retains legacy JupyterLab 3 instructions.

Local execution and cost: The notebook interface runs locally. The README documents local Gremlin Server, Blazegraph, and Neo4j connections, so AWS is not mandatory. Selected lessons are written around Neptune and may require loading/configuration changes for other engines. Neptune-specific, ML, or analytics examples require AWS services and may incur charges.

Reliability engineering takeaway: Make query scope and result semantics explicit. As a book extension, add a missing edge and a duplicate path, then check how the answer changes under the same traversal budget.

Limitations: Query visualization does not verify identity, provenance, or authorization. Notebook magics and seed commands vary by backend; support for a protocol does not mean every Neptune example transfers unchanged. The dependency and README Python ranges differ, so select one versioned installation path.

License and update check: Apache-2.0 (`LICENSE`); third-party notices are separate. Latest substantive change identified: 2026-08-03; see the verification record for its scope and later maintenance activity.

State, Tool Use, and Orchestration

Hugging Face Agents Course

Source: huggingface/agents-course
Type: Educational course.
Maintainer: Hugging Face; course credits name Ben Burtenshaw, Joffrey Thomas, Thomas Simonini, and Sergio Paniego.

What you will learn: Describe a callable tool, observe how the description reaches the model, and follow the action/observation cycle in a small implementation before adopting a framework.

Book connection: 05.02 — Tool Schemas and Function Calling.

Recommended lessons:

- Unit 1: Tools.
- Unit 1: Dummy Agent Library.

Prerequisites: Basic Python and LLM concepts. The dummy-agent lesson uses `huggingface_hub.InferenceClient` and an HF token; later units add their own framework dependencies.

Local execution and cost: Python can execute locally, but the selected dummy-agent lesson calls hosted inference. Course access is free; model access, provider credits, quotas, and optional hosted Spaces are separate. A fully local model would require adapting the example.

Reliability engineering takeaway: Trace the boundary between a proposed operation, host execution, and returned observation. As a book extension, reject an unknown tool and malformed arguments before execution.

Limitations: The dummy library is explicitly pedagogical. A function description does not authenticate the caller, enforce permissions, or sandbox generated code. Hosted models and credentials can change; a completed course or leaderboard result does not certify a production workflow.

License and update check: Apache-2.0 (`LICENSE`); externally linked models and datasets have separate terms. Latest substantive change identified: 2026-04-23; see the verification record for its scope and later maintenance activity.

LangGraph 101

Source: langchain-ai/langgraph-101
Type: Instructor-oriented educational notebooks.
Maintainer: LangChain.

What you will learn: Represent conversation state, pause a proposed operation for review, and resume with approval, rejection, or edited arguments. Inspect middleware as part of the host's control flow.

Book connection: 13.02 — Harnesses and State Machines.

Recommended lessons:

- 101: Adding Memory & State.
- 102: interrupts, rejection, and editing before execution.

Prerequisites: Python >=3.11, `uv`, Jupyter, LangChain, LangGraph, and provider SDKs. Use the repository's lockfile with the selected revision. Advanced lessons introduce additional services and packages.

Local execution and cost: Notebooks and the development server run locally, but the current `utils/models.py` defaults to Anthropic's hosted Claude Haiku 4.5. A provider account and usage budget are needed as written. Local inference would require replacing the model configuration; hosted tracing is a separate choice.

Reliability engineering takeaway: An approval is workflow state tied to an exact proposed effect. As a book extension, change the arguments after approval and require revalidation before dispatch.

Limitations: The README still describes OpenAI o3-mini as the default, unlike the code. `MemorySaver` demonstrates in-process checkpointing, not crash-durable storage. Interrupt/resume does not guarantee exactly-once external effects. Recent dependency updates do not by themselves establish that all lessons still run.

License and update check: MIT (`LICENSE`). Latest substantive change identified: 2026-04-28; see the verification record for its scope and later maintenance activity.

Authorization and Security

Open Policy Agent

Source: open-policy-agent/opa-sdk-demos
Type: Framework documentation and reference integration demos.
Maintainer: Open Policy Agent project (CNCF).

What you will learn: Pass structured subject, action, and resource information to a policy decision point, then enforce the returned decision in the application. Read the OPA documentation and Rego language guide alongside the integration.

Book connection: 11.02 — RBAC, ABAC, Delegation, and Capabilities.

Recommended lessons:

- NestJS: request attributes, AuthzGuard, and response mapping.
- NestJS cats policy.
- Policy Testing: write allow and deny cases.

Prerequisites: JSON, HTTP, Rego basics, an OPA executable, and Node/TypeScript for the chosen NestJS demo. Other demos use Java, C#, React, or Docker Compose.

Local execution and cost: The selected NestJS README documents a local OPA server and local application; no LLM, paid API, or cloud account is required for that path. Other demos use different container images, including EOPA, whose deployment requirements need separate review.

Reliability engineering takeaway: Authenticate policy inputs and enforce decisions at the protected service. Add denied, missing-attribute, undefined-decision, and unavailable-policy-service cases; require explicit allow before the protected operation.

Limitations: Demo request headers and bodies are not trusted identity sources. The cats policy allows get/list broadly, so it is not a tenant-isolation policy. TicketHub shows an illustrative identity cookie and admin-oriented denial reasons. Frontend visibility is not backend enforcement. Test the installed OPA/Rego version rather than copying older syntax blindly.

License and update check: Apache-2.0 for the SDK demos (`LICENSE`). Policy-engine, documentation, and optional deployment components should be checked at their own versions. Latest substantive change identified: 2025-09-16; see the verification record for its scope and later maintenance activity.

Agentic Security

Source: luisalima/agentic-security
Type: Educational guide and reference defense implementations.
Maintainer: Luisa Lima.

What you will learn: Locate where untrusted content can influence a privileged operation. Inspect examples of poisoned tool descriptions, allowlists, constrained data exchange, and validation before execution.

Book connection: 11.04 — Retrieval and Execution Boundaries.

Recommended lessons:

- Threat model: map data, egress, and trust boundaries.
- Tool and MCP manifest validation notebook.
- Secure architecture: dual-model and typed extraction patterns.

Prerequisites: Python >=3.10, `uv`, Marimo, and Pydantic; core dependencies include OpenAI and Anthropic clients. Model-backed examples can use a configured provider or the documented Ollama setup.

Local execution and cost: The selected manifest-validation notebook is local and does not need paid inference. Model-backed demonstrations need either local model weights and compute or hosted API credentials and potential charges. Infrastructure exercises require their own sandbox/network setup.

Reliability engineering takeaway: Evaluate what a compromised proposal could actually cause. As a book extension, bypass the description scanner and verify that the executor still rejects an out-of-scope operation.

Limitations: The project labels itself a work in progress and focuses on prompt injection. Detection heuristics, dual-model separation, and typed extraction are not universal guarantees. Its bundled attack examples are not independent evidence of security. The isolation guide's named Docker network is not an egress policy by itself.

License and update check: MIT (`LICENSE`). Latest substantive change identified: 2026-07-02; see the verification record for its scope and later maintenance activity.

Observability and Evaluation

Arize Phoenix

Source: Arize-ai/phoenix
Type: Observability framework and documentation with tutorials.
Maintainer: Arize AI.

What you will learn: Connect instrumentation to a trace collector, inspect nested operations, and associate datasets and evaluator results with experiments. Keep the identity of the run and its evidence visible across stages.

Book connection: 16.01 — Tracing the Context-to-Outcome Pipeline.

Recommended lessons:

- Python tracing quickstart: model, tool, and retrieval spans.
- Datasets and experiments quickstart.

Prerequisites: Python, an instrumentable application, Phoenix, OpenTelemetry/OpenInference packages, and a notebook environment. The inspected Phoenix package specifies Python >=3.11,<3.15.

Local execution and cost: The collector and UI can be self-hosted locally. Both selected notebooks call OpenAI models, and the tracing example also calls embeddings, so those examples require API access and a usage budget. Local collection does not imply local model inference.

Reliability engineering takeaway: Record retrieval and tool boundaries explicitly. Apply the chapter's content-capture policy before exporting spans; telemetry can otherwise become another copy of protected context.

Limitations: The repository license is source available with restrictions, despite broad open-source wording in its README. Automatic instrumentation does not capture every business decision, and an LLM evaluator needs calibration. Tutorial content capture and local defaults are not a production retention, access-control, or availability design.

License and update check: Elastic License 2.0 (ELv2) at the repository root. This includes hosted-service restrictions; do not label it MIT or Apache licensed. Individual SDK packages have their own license files. Latest substantive change identified: 2026-10-12; see the verification record for its scope and later maintenance activity.

LLM Observatory

Source: fraps93/llm-observatory
Type: Educational two-notebook demonstration.
Maintainer: GitHub maintainer fraps93.

What you will learn: Follow one small RAG pipeline through OpenTelemetry exporters, then compare DeepEval and a custom judge on the same outputs. Inspect disagreement rather than collapsing evaluator results into one score.

Book connection: 17.01 — Evals and Benchmarks.

Recommended lessons:

- 01: Multi-backend tracing.
- 02: Eval frameworks and disagreement analysis.

Prerequisites: Python, Jupyter, Ollama and downloaded model weights, ChromaDB, sentence-transformers, Phoenix, OpenTelemetry, and DeepEval. The README suggests Python 3.10+, but broadly constrained dependencies can now require newer Python.

Local execution and cost: Generation and judging use Ollama locally. Phoenix runs locally; Langfuse and Opik cloud exporters are optional and need accounts/credentials. No paid inference is required by the supplied local path, but model downloads and sufficient memory are needed. Cloud free tiers are not a lasting pricing guarantee.

Reliability engineering takeaway: A disagreement between judges is a case to inspect. Compare both with human judgments and preserve parse failures or missing scores as evaluation failures.

Limitations: Only one initial commit was visible. The README calls dependencies pinned, but requirements mostly use lower bounds or no bounds. Ten synthetic questions cannot establish evaluator superiority. The reported RAGAS difficulties are this project's observations, not general evidence that RAGAS requires a proprietary model. Missing cloud credentials can silently remove an exporter.

License and update check: No repository-wide license found in the seven-file tree or README; reuse permission is unconfirmed. Latest substantive change identified: 2026-05-09; see the verification record for its scope and later maintenance activity.

Bring the Exercise Back to the System

After one lesson, return to the chapter's contract. Identify the source of authority, the state that survives failure, the validation boundary, and the observable outcome. Add one controlled failure and explain which component owns it. This turns a tutorial into practice in building systems that can be inspected, corrected, and evaluated.

Return to the book outline